

カフェ在庫管理システム

設計仕様書

Version: 1.3

作成日: 2026/08/18

作成者: プサディーくんパイサン ナッタチャイ.

1. システム概要

1.1. システム名

カフェ在庫管理システム

1.2. 目的

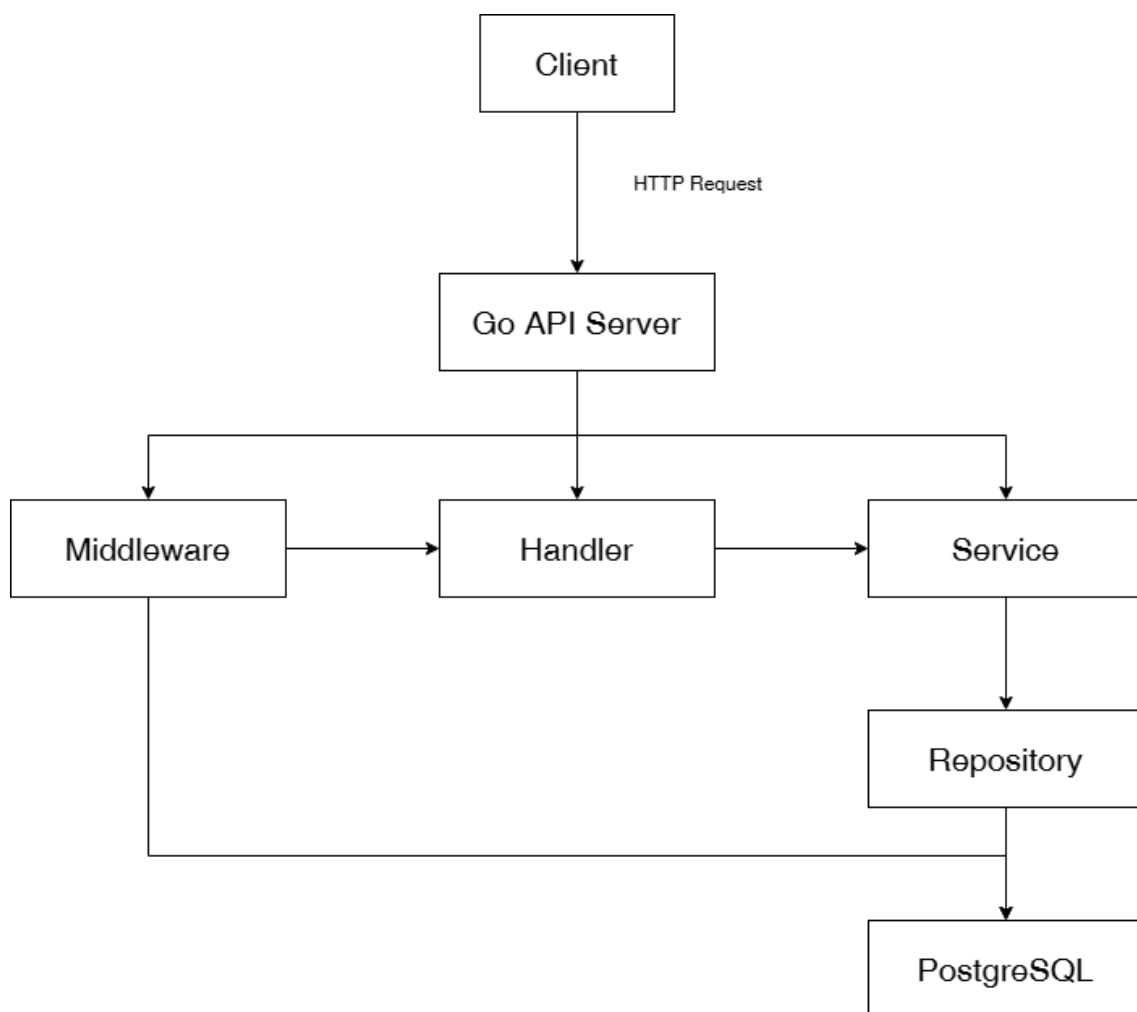
Go（`net/http`）と PostgreSQL で構築された、複数店舗展開のカフェチェーンを管理するための本番想定 RESTful バックエンド API。

このシステムは、単一ブランドの下で複数のカフェ店舗を運営する企業向けに設計されています。本部はカフェ、商品、ユーザー、在庫を一元管理でき、各店舗のスタッフはリアルタイムで在庫数を更新できます。また、顧客が近くのカフェや在庫のある商品を見つけられるよう、位置情報ベースの検索機能も提供します。

1.3. 開発環境

項目	内容
言語	Go
HTTP サーバ	net/http
データベース	PostgreSQL
データベースドライバ	pgx
認証	JWT
パスワードハッシュ化	bcrypt
コンテナ	Docker
構成	Layered Architecture
API スタイル	RESTful API
インフラ	AWS ECS
バージョン管理	Git & GitHub

2. システム構成



3.機能一覧

機能 ID	機能名	概要	権限
CAFE-01	カフェ登録	新しいカフェを登録する	ADMIN
CAFE-02	カフェ詳細取得	指定したカフェの詳細情報を取得する	全ユーザー
CAFE-03	カフェ一覧取得	カフェ一覧を取得する	全ユーザー
CAFE-04	カフェ更新	カフェ情報を更新する	ADMIN
CAFE-05	カフェ削除	カフェ情報を削除する	ADMIN
CATE-01	カテゴリ登録	新しいカテゴリを登録する	ADMIN
CATE-02	カテゴリ詳細取得	指定したカテゴリの詳細を取得する	全ユーザー
CATE-03	カテゴリ一覧取得	カテゴリ一覧を取得する	全ユーザー
CATE-04	カテゴリ更新	カテゴリ情報を更新する	ADMIN
CATE-05	カテゴリ削除	カテゴリ情報を削除する	ADMIN
PROD-01	商品登録	新しい商品を登録する	ADMIN
PROD-02	商品詳細取得	商品詳細を取得する	全ユーザー
PROD-03	商品一覧取得	カフェの商品一覧を取得する	全ユーザー

PROD-04	商品更新	商品情報を更新する	ADMIN
PROD-05	商品削除	商品情報を削除する	ADMIN
INV-01	在庫取得	商品の現在在庫数を取得する	全ユーザー
INV-02	在庫一覧取得	在庫一覧を取得する	全ユーザー
INV-03	在庫更新	在庫数量を更新する	STAFF / ADMIN
INV-04	在庫履歴取得	在庫変更履歴を取得する	STAFF / ADMIN
USER-01	ユーザー登録	新しいユーザーを登録する	ADMIN
USER-02	ユーザー詳細取得	指定したユーザーの詳細情報を取得する	ADMIN
USER-03	ユーザー一覧取得	ユーザー一覧を取得する	ADMIN
USER-04	ユーザー更新	ユーザー情報を更新する	ADMIN
USER-05	ユーザー削除	ユーザー情報を削除する	ADMIN
AUTH-01	ログイン	メールアドレスとパスワードで認証し、 JWT を発行する	全ユーザー

4.DB 設計

4.1. cafes

項目	型	PK	FK	NULL	説明
id	SERIAL	○		×	カフェ ID
name	VARCHAR(255)			×	カフェ名
address	TEXT			×	住所
latitude	DECIMAL(9,6)			×	緯度
longitude	DECIMAL(9,6)			×	経度
nearest_station	VARCHAR(255)			○	最寄り駅
opening_time	TIME			○	開店時間
closing_time	TIME			○	閉店時間
created_at	TIMESTAMP			×	作成日時
updated_at	TIMESTAMP			×	更新日時

4.2. products

項目	型	PK	FK	NULL	説明
id	SERIAL	○		×	商品 ID
cafe_id	INT		○	×	カフェ ID
category_id	INT		○	×	カテゴリ ID
name	TEXT			×	商品名
description	TEXT			○	商品説明
price	NUMERIC(10,2)			○	価格
is_active	BOOLEAN			×	有効フラグ
created_at	TIMESTAMP			×	作成日時
updated_at	TIMESTAMP			×	更新日時

4.3. categories

項目	型	PK	FK	NULL	説明
id	SERIAL	○		×	カテゴリ ID
name	VARCHAR(100)			×	カテゴリ名
created_at	TIMESTAMP			×	作成日時
updated_at	TIMESTAMP			×	更新日時

備考

name は UNIQUE 制約により、カテゴリ名は重複してはいけません。

4.4. inventory

項目	型	PK	FK	NULL	説明
id	SERIAL	○		×	在庫 ID
product_id	INT		○	×	商品 ID
stock_quantity	INT			×	在庫数量
created_at	TIMESTAMP			×	作成日時
updated_at	TIMESTAMP			×	更新日時

備考

- product_id は UNIQUE 制約により、1 商品につき 1 つの在庫情報を保持する。
- 在庫数は更新されるたびに inventory_logs に履歴を記録する。
- stock_quantity は 0 未満にしてはいけません。

4.5. users

項目	型	PK	FK	NULL	説明
id	SERIAL	○		×	ユーザーID
username	VARCHAR(100)			×	ユーザー名
email	VARCHAR(255)			×	メールアドレス
password_hash	TEXT			×	ハッシュ化されたパスワード
user_role	VARCHAR(10)			×	権限
is_active	BOOLEAN			×	有効フラグ
created_at	TIMESTAMP			×	作成日時
updated_at	TIMESTAMP			×	更新日時

備考

email は UNIQUE 制約により、メールアドレスは重複してはいけません。

user_role (constraint)

値	説明
ADMIN	管理者
STAFF	スタッフ

4.6. inventory_logs

項目	型	PK	FK	NULL	説明
id	SERIAL	○		×	ログ ID
inventory_id	INT		○	×	在庫 ID
user_id	INT		○	×	ユーザーID
change_quantity	INT			×	変更数量
operation	VARCHAR(6)			×	操作種別
created_at	TIMESTAMP			×	作成日時

operation (constraint)

値	説明
IN	入庫
OUT	出庫
ADJUST	在庫調整

5.API 設計

5.1. カフェ登録

Endpoint (認証:必須)

POST /api/v1/cafes

Request

```
{  
  "name": "Tokyo Cafe",  
  "address": "Shinjuku",  
  "latitude": 35.6580,  
  "longitude": 139.7016,  
  "nearest_station": "Shinjuku station",  
  "opening_time": "08:00",  
  "closing_time": "21:00"  
}
```

Response

```
{  
  "id": 1,  
  "message": "Cafe created successfully"  
}
```

5.2. カフェ詳細取得

Endpoint (認証 : 不要)

```
GET /api/v1/cafes/{id}
```

Response

```
{  
  
  "id": 1,  
  
  "name": "Tokyo Cafe",  
  
  "address": "Shinjuku",  
  
  "latitude": 35.6580,  
  
  "longitude": 139.7016,  
  
  "nearest_station": " Shinjuku station",  
  
  "opening_time": "08:00",  
  
  "closing_time": "21:00"  
  
  "created_at": "2006-01-02 15:04:05",  
  
  "updated_at": "2006-01-02 15:04:05"  
  
}
```


5.3. カフェ一覧取得

Endpoint (認証 : 不要)

GET /api/v1/cafes

Query Parameters

項目	タイプ	説明
station	string	最寄り駅
limit	int	最大表示レコード
lat	float64	検索基準の緯度
lng	float64	検索基準の経度
radius	float64	lat/lng からの検索半径

Response

```
[
  {
    "id": 1,
    "name": "Tokyo Cafe",
    "address": "Shinjuku",
    "latitude": 35.6580,
    "longitude": 139.7016,
    "nearest_station": " Shinjuku station",
    "opening_time": "08:00",
    "closing_time": "21:00"
    "created_at": "2026-01-02 15:04:05",
    "updated_at": "2026-01-02 15:04:05"
  }
]
```

5.4. カフェ更新

Endpoint (認証 : 必須)

```
PATCH /api/v1/cafes/{id}
```

5.5. カフェ削除

Endpoint (認証 : 必須)

```
DELETE /api/v1/cafes/{id}
```

Error response

409 Conflict

```
{
```

```
  "error": "cannot delete: other records still depend on this
```

```
item"
```

```
}
```

5.6. カテゴリ登録

Endpoint (認証:必須)

```
POST /api/v1/categories
```

Request

```
{  
  
  "name": "Drink",  
  
}
```

Response

```
{  
  
  "id": 1,  
  
  "message": "Category created successfully"  
  
}
```

Error response

409 Conflict

{

"error": "this category name is already used: <name>"

}

5.7. カテゴリ詳細取得

Endpoint (認証 : 不要)

```
GET /api/v1/categories/{id}
```

Response

```
{  
  "id": 1,  
  "name": "Drink",  
  "created_at": "2006-01-02 15:04:05",  
  "updated_at": "2006-01-02 15:04:05"  
}
```

5.8. カテゴリ一覧取得

Endpoint (認証 : 不要)

GET /api/v1/categories

Query Parameters

項目	タイプ	説明
limit	int	最大表示レコード

Response

```
[  
  {  
    "id": 1,  
    "name": "Drink",  
    "created_at": "2026-01-02 15:04:05",  
    "updated_at": "2026-01-02 15:04:05"  
  }  
]
```


5.9. カテゴリ更新

Endpoint (認証 : 必須)

```
PATCH /api/v1/categories/{id}
```

Error response

409 Conflict

```
{
```

```
  "error": "this category name is already used: <name>"
```

```
}
```

5.10. カテゴリ削除

Endpoint (認証 : 必須)

```
DELETE /api/v1/categories/{id}
```

Error response

409 Conflict

```
{
```

```
  "error": "cannot delete: other records still depend on this
```

```
  item"
```

```
}
```

5.11. 商品登録

Endpoint (認証 : 必須)

```
POST /api/v1/products
```

Request

```
{  
  "cafe_id": 1,  
  "category_id": 1,  
  "name": "Latte",  
  "description": "Hot latte",  
  "price": 480.00  
}
```

Response

```
{  
  "id": 1,  
  "message": "Product created successfully"  
}
```

5.12. 商品詳細取得

Endpoint (認証 : 不要)

```
GET /api/v1/products/{id}
```

Response

```
{
  "id": 1,
  "cafe_id": 1,
  "cafe_name": "Tokyo Cafe",
  "category_id": 1,
  "category_name": "Drink",
  "name": "Latte",
  "description": "Hot Latte",
  "price": 480.00,
  "created_at": "2026-01-02 15:04:05",
  "updated_at": "2026-01-02 15:04:05"
}
```

5.13. 商品一覧取得

Endpoint (認証 : 不要)

```
GET /api/v1/cafes/{cafeId}/products
```

Response

```
[  
  {  
    "id": 1,  
    "cafe_id":1,  
    "cafe_name": "Tokyo Cafe",  
    "category_id":1,  
    "category_name": "Drink",  
    "name": "Latte",  
    "description": "Hot Latte",  
    "price": 480.00,  
    "created_at": "2006-01-02 15:04:05",  
    "updated_at": "2006-01-02 15:04:05"  
  }  
]
```

Query Parameters

項目	タイプ	説明
limit	int	最大表示レコード

5.14. 商品更新

Endpoint (認証 : 必須)

```
PATCH /api/v1/products/{id}
```

5.15. 商品削除

Endpoint (認証 : 必須)

```
DELETE /api/v1/products/{id}
```


5.16. 在庫取得

Endpoint (認証 : 不要)

```
GET /api/v1/inventory/{productId}
```

Response

```
{  
  "product_name": "Matcha cake",  
  "cafe_id": 1,  
  "cafe_name": "Tokyo cafe",  
  "stock_quantity": 50,  
  "updated_at": "2006-01-02 15:04:05"  
}
```

5.17. 在庫一覧取得

Endpoint (認証 : 不要)

GET /api/v1/inventory

Query Parameters

項目	タイプ	説明
name	string	商品名
limit	int	最大表示レコード

Response

```
[  
  {  
    "id": 1,  
    "product_id": 1,  
    "product_name": "Matcha Cake",  
    "cafe_id": 1,  
    "cafe_name": "Tokyo cafe",  
    "stock_quantity": 40,  
    "updated_at": "2006-01-02 15:04:05"  
  },  
]
```

5.18. 在庫更新

Endpoint (認証 : 必須)

```
PATCH /api/v1/inventory/{productId}
```

Request

```
{  
  "change_quantity": 5,  
  "operation": "OUT"  
}
```

処理フロー

1. inventory.stock_quantity を更新する。
2. inventory_logs に操作履歴を登録する。
3. 更新後の最新在庫数量を返却する。

Response

```
{  
  
  "message": "Inventory updated",  
  
  "stock_quantity": 45  
  
}
```

Request (Adjust)

```
{  
  
  "change_quantity": 5,  
  
  "operation": "ADJUST"  
  
}
```

Response

```
{  
  
  "message": "Inventory updated",  
  
  "stock_quantity": 5  
  
}
```

備考

ADJUST の場合、change_quantity は差分ではなく更新後の在庫数量そのもの（絶対値）を表す。上記の例では、更新前の在庫数量に関わらず 5 に設定される。

Error response

409 Conflict

{

"error": "the stock doesnt have enough items"

}

5.19. 在庫履歴一覧取得

Endpoint (認証 : 必須)

GET /api/v1/inventory/logs

Query Parameters

項目	タイプ	説明
inventory_id	int	在庫 ID
user_id	int	ユーザーID
operation	string	操作種別
from	date	最初日
to	date	最終日
limit	int	最大表示レコード

Response

```
[  
  {  
    "id": 1,  
    "inventory_id": 1,  
    "user_id": 1,  
    "operation": "OUT",  
    "change_quantity": 5,  
    "created_at": "2006-01-02 15:04:05"  
  },  
]
```


5.20. ユーザー登録

Endpoint (認証 : 必須)

```
POST /api/v1/users
```

Request

```
{  
  "username": "staff01",  
  "email": "staff@example.com",  
  "password": "password01",  
  "user_role": "STAFF"  
}
```

Response

```
{  
  
  "id": 1,  
  
  "message": "User created successfully"  
  
}
```

Error response

```
409 Conflict  
  
{  
  
  "error": "this email is already used: <email>"  
  
}
```

5.21. ユーザー詳細取得

Endpoint (認証 : 必須)

```
GET /api/v1/users/{id}
```

Response

```
{  
  
  "id": 1,  
  
  "username": "staff01",  
  
  "email": "staff@example.com",  
  
  "user_role": "STAFF",  
  
  "is_active": true,  
  
  "created_at": "2026-01-02 15:04:05",  
  
  "updated_at": "2026-01-02 15:04:05"  
}
```

5.22. ユーザー一覧取得

Endpoint (認証 : 必須)

```
GET /api/v1/users
```

Response

```
[
  {
    "id": "1",
    "username": "staff01",
    "email": "staff@example.com",
    "user_role": "STAFF",
    "is_active": true,
    "created_at": "2026-01-02 15:04:05",
    "updated_at": "2026-01-02 15:04:05"
  }
]
```

Query Parameters

項目	タイプ	説明
limit	int	最大表示レコード

5.23. ユーザー更新

Endpoint (認証 : 必須)

```
PATCH /api/v1/users/{id}
```

Error response

409 Conflict

```
{
```

```
  "error": "this email is already used: <email>"
```

```
}
```

5.24. ユーザー削除

Endpoint (認証 : 必須)

```
DELETE /api/v1/users/{id}
```

5.25. ユーザー認証

Endpoint (認証:不要)

```
POST /api/v1/auth/login
```

Request

```
{  
  
  "email": "admin@example.com",  
  
  "password": "password01"  
}
```

Response

```
{  
  
  "token": "jwt_token"  
}
```

Error response

401 Unauthorized

{

"error": "invalid email or password"

}

Error response

401 Unauthorized

{

"error": "this user account has been deactivated"

}

HTTP ステータスコード

Status	説明
200 OK	正常終了
201 Created	登録成功
204 No Content	削除成功
400 Bad Request	リクエスト不正
401 Unauthorized	認証エラー
403 Forbidden	権限不足
404 Not Found	リソースが存在しない
409 Conflict	データ重複・競合
500 Internal Server Error	サーバ内部エラー

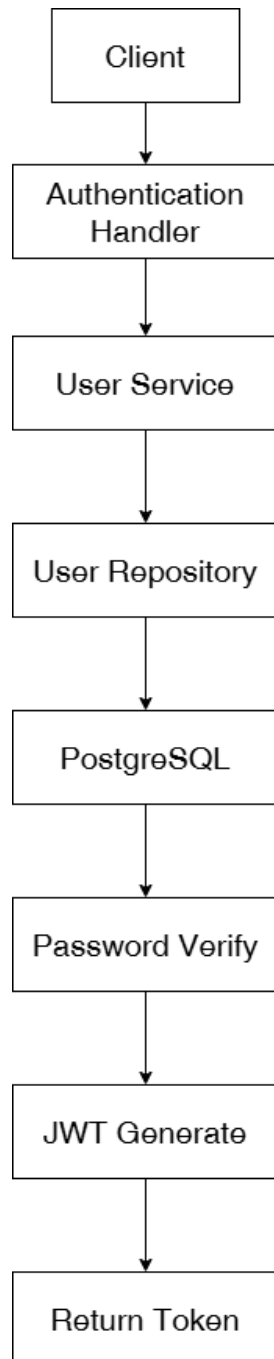
6.認証設計

6.1. 認証方式

本システムでは、JWT（JSON Web Token）認証を採用する。

ユーザーはメールアドレスとパスワードでログインし、認証に構成すると JWT を発行する。以降の認証が必要な API では、HTTP リクエストヘッダーに JWT を設定してアクセスする。

6.2. 認証フロー



6.3. 認証対象 API

API	Method	認証
/cafes	POST	ADMIN
/cafes/{id}	GET	不要
/cafes	GET	不要
/cafes/{id}	PATCH	ADMIN
/cafes/{id}	DELETE	ADMIN
/categories	POST	ADMIN
/categories/{id}	GET	不要
/categories	GET	不要
/categories/{id}	PATCH	ADMIN
/categories/{id}	DELETE	ADMIN
/products	POST	ADMIN
/products/{id}	GET	不要
/products	GET	不要
/products/{id}	PATCH	ADMIN

/products/{id}	DELETE	ADMIN
/inventory/{productId}	GET	不要
/inventory	GET	不要
/inventory/{productId}	PATCH	STAFF/ADMIN
/inventory/logs	GET	STAFF/ADMIN
/users	POST	ADMIN
/users/{id}	GET	ADMIN
/users	GET	ADMIN
/users/{id}	PATCH	ADMIN
/users/{id}	DELETE	ADMIN
/auth/login	POST	不要

6.4. 認証対象外

認証失敗

HTTP Status

401 Unauthorized

Response

```
{  
  
  "error": "Invalid or expired token"  
  
}
```

権限不足

HTTP Status

403 Forbidden

Response

```
{  
  "error": "Permission denied"  
}
```

7.ディレクトリ構成

```
cafe-inventory-api/  
├─ cmd/  
│   └─ server/  
│       └─ main.go  
├─ internal/  
  
│   ├─ handler/                // HTTP リクエスト処理  
  
│   ├─ service/                // ビジネスロジック  
  
│   ├─ repository/            // PostgreSQL アクセス  
  
│   ├─ model/                  // ドメインモデル  
  
│   ├─ middleware/            // 共通ミドルウェア  
  
│   ├─ router/                 // ルーティング設定  
  
│   └─ database/              // DB 接続  
  
│   └─ utils/                  // 共通ユーティリティ  
  
├─ migrations/                 // SQL マイグレーション  
  
├─ .env  
├─ .gitignore  
├─ docker-compose.yml  
├─ go.mod  
├─ go.sum  
└─ README.md
```